

NDN-Python-Repo

Current Status and Future Directions

Zhaoning Kong, Zixuan Zhong, Alex Feltus, Susmit Shannigrahi, Lixia Zhang

Motivation

- Closing the loop
 - Large science datasets (such as High Energy Physics and Genomics) can benefit from NDN's name-based, location transparent networking
 - We use the genomics example in this talk
- A critical aspect is storage
 - `ndn-python-repo` provides persistent storage

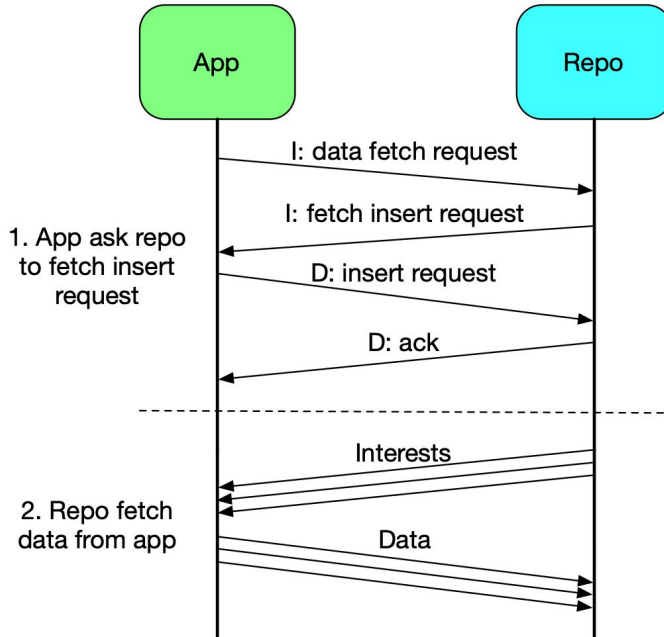
Genomics Storage Requirements

- Should be able to support very large datasets
 - TBs to PBs (We have tested the repo with over 1TB data)
- Provide persistent packet storage
 - at a file level
- Provides transparent data access for the requesters
- Replicated for availability and supporting geographically distributed users

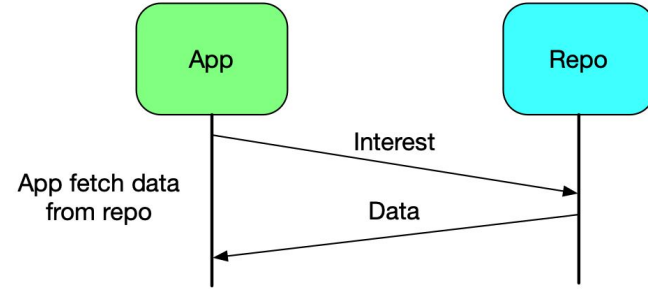
Genomics use case

- Allow the publisher(s) to publish datasets into the repo
 - Under community defined names
- Data is automatically replicated
 - Degree of replication and location - defined by use cases
 - Automatically maintains the degree of replication when failure happens
- Requester looks up the content name (e.g., using a separate catalog) and retrieves it using names

NDN Repo Overview - Publication and Access Interface



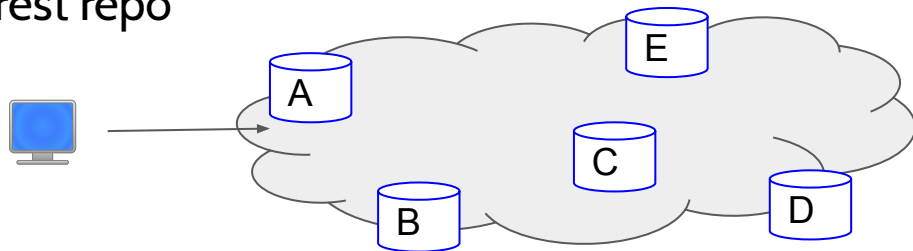
1) Publish data into repo



2) Fetch data from repo

NDN Repo - Replication

- Deploying a set of distributed repo nodes
 - Improve availability and performance
 - Utilize anycast: all repo nodes make the same routing announcement, data fetching interests forwarded to nearest repo



- Replicating stored data files
 - Avoid single point of failure
 - Repos use consistent hashing to decide which file replicated where

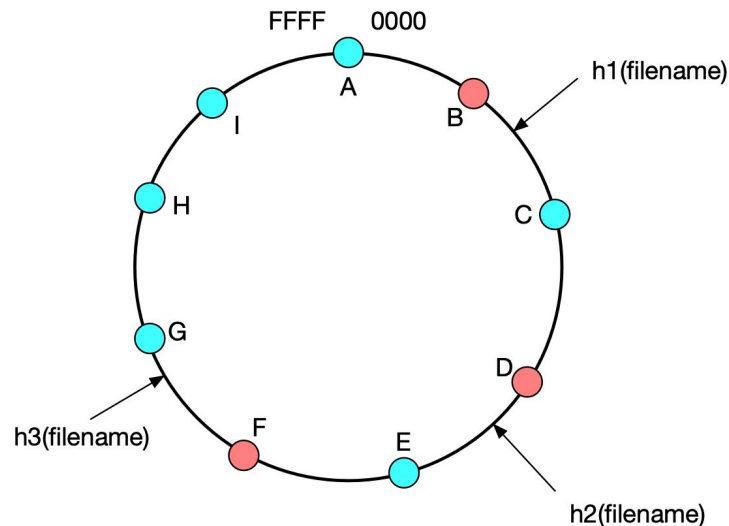
Repo Replication with Consistent Hashing

Consistent hashing

- Hash both repo node and filename onto a ring
- Each file copy will be stored by the closest node smaller than it

Multiple hash functions

- Use multiple hash functions, to store file onto multiple nodes



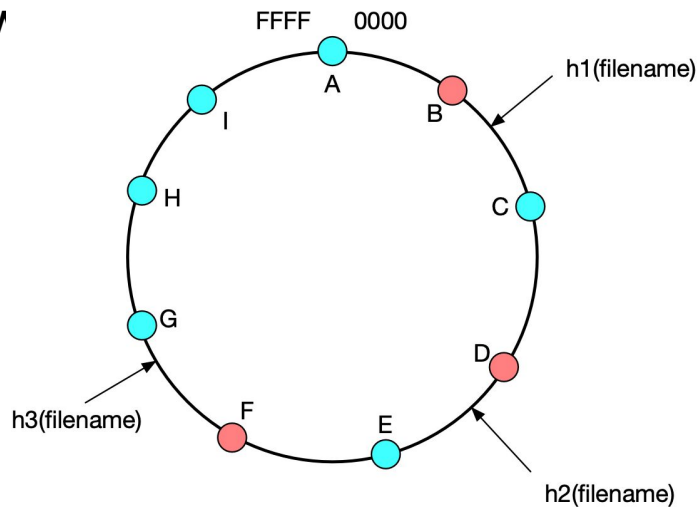
Membership Management

To construct such a ring, each repo node needs a **global view** of:

1. A list of all other members
2. Know newly added nodes quickly
3. Know crashed nodes quickly

How?

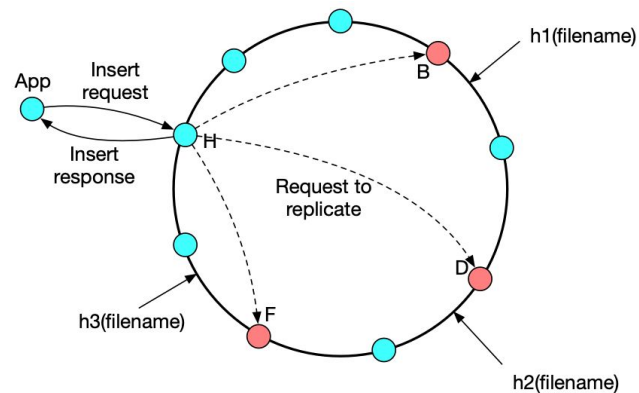
1. Each node periodically ping its next neighbor (ping contains member list with timestamp)
2. Broadcast the failure for faster failure discovery
3. Use a predefined namespace for new nodes to ask to join (e.g., /repo_group1/join)



Insert / Retrieve Data

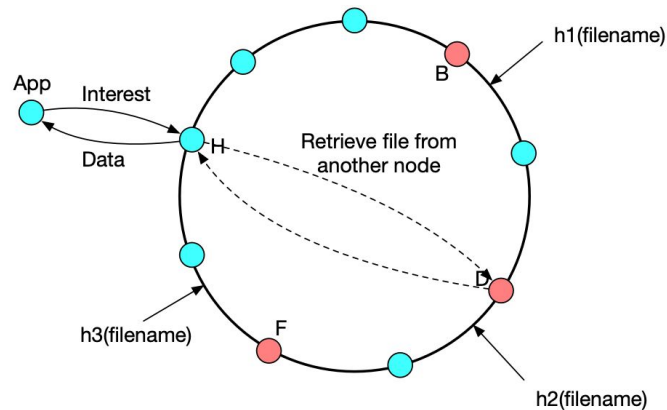
Insert data into repo

1. App anycast insert request to nearest neighbor (e.g. H)
2. H looks at the hash ring, and decide to insert the file to B, D, and F



Retrieve data from repo

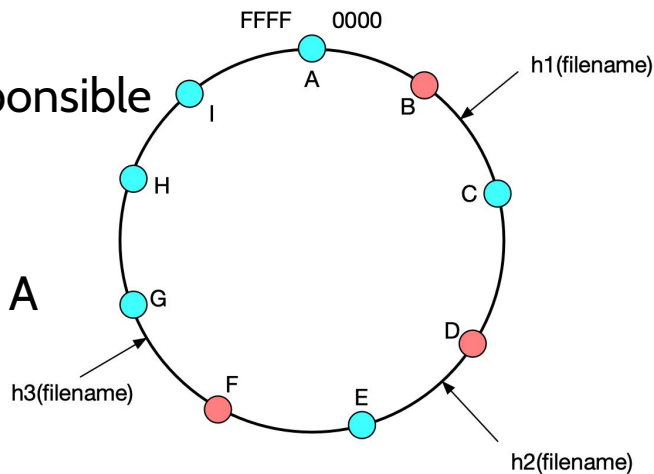
1. Interest sent to a repo (e.g. H)
2. H doesn't have the file, so retrieves from another node that has it (e.g. D) and send it back to the app



Respond to membership changes

Existing nodes fail

1. For each file, the next clockwise node is responsible for failure recovery
2. E.g, node D finds B crashed, D looks at its common files with B, and replicate them on A



New nodes join

1. For each file, the next clockwise node is responsible for re-distributing the file
2. E.g. node F joins, G looks at its own files, and decide that some need to be moved to F

Future Work

- Enhance the repo with file semantics
- Update the repo protocol spec
- Implement the repo replication
- Publish a tech report on the python repo design and implementation